Luckfox Image Builder

Программная система автоматизированной сборки
Linux-образов для микрокомпьютеров Luckfox Pico

Техническая документация проекта

Команда 44 · Жданов, Поздняков
Курсовой проект

Содержание

1. Назначение и предметная область
2. Технологический стек
3. Архитектура системы
4. Структура проекта (что где находится)
5. Модель данных и целостность
6. Серверная часть (backend)
7. REST API
8. Очередь сборок и разрешение зависимостей
9. Клиентская часть (frontend)
10. Тулчейн сборки Luckfox (симуляция и реальная)
11. Жизненный цикл образа
12. Развёртывание и запуск
13. Учётные данные и проверка работоспособности
14. Статус проекта и что доработать

1. Назначение и предметная область

Luckfox Image Builder — распределённая веб-система, позволяющая удобно конструировать и удалённо собирать кастомные Linux-образы для микрокомпьютеров Luckfox Pico (платформа Rockchip RV1103 / RV1106). Система избавляет пользователя от необходимости локально разворачивать тяжёлую среду сборки (buildroot), долго настраивать её и тратить ресурсы и место на диске.

Компания, выпускающая оборудование для видеонаблюдения и компьютерного зрения, перешла на микрокомпьютеры Luckfox. Штатный образ вендора не покрывает требований, поэтому нужно собирать множество уникальных образов. Система предоставляет каталог готовых образов и компонентов, визуальный редактор образов с автоматическим разрешением зависимостей, удалённую сборку и выдачу готового .img-файла.

Ключевые сущности

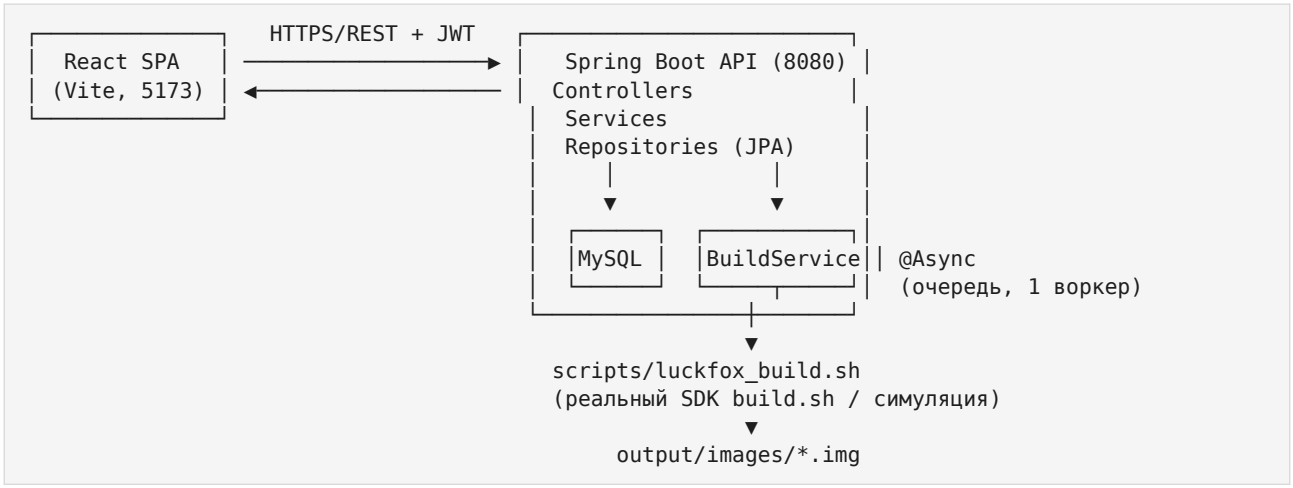
- **User** — пользователь системы (клиент или администратор).
- **Image** — собираемый образ ОС с заданным статусом сборки.
- **Component** — пакет или библиотека; имеет рекурсивные зависимости от других компонентов.
- **ComponentType** — тип компонента (Пакет / Библиотека).
- **Role / Permission** — роли и права доступа (USER, ADMIN).

2. Технологический стек

Слой	Технология	Назначение
Backend	Java 17, Spring Boot 3 (Web, Data JPA, Security, Validation)	REST API, авторизация JWT, оркестрация сборки
СУБД	MySQL 8	Хранение пользователей, образов, компонентов и связей
Frontend	React + Vite, React Router, Zustand, Axios	SPA: библиотеки, редактор образов и компонентов
Воркеры сборки	Docker + Bash	Изолированная сборка образов через SDK Luckfox
Тулчейн	Luckfox Pico SDK (Rockchip), кросс-компилятор	Реальная сборка прошиваемых образов
Нагруз. тест.	Apache JMeter	Проверка НФТ (план)

3. Архитектура системы

Система состоит из трёх контейнеров: SPA-фронтенд, REST-бэкенд и база данных. Бэкенд при заказе сборки асинхронно вызывает обёртку тулчейна, которая порождает .img-артефакт.



Слои бэкенда строго разделены: **controller** (HTTP/DTO) → **service** (бизнес-логика, транзакции, разрешение зависимостей, оркестрация) → **repository** (DAO на Spring Data JPA) → **entity** (маппинг таблиц). Наружу отдаются только DTO — это исключает рекурсию графа зависимостей и проблемы ленивой загрузки.

4. Структура проекта (что где находится)

```
course-work-team-44-zhdanov-pozdnyakov/
├── backend/                Spring Boot 3 (Java 17)
│   ├── src/main/java/com/luckfox/imagebuilder/
│   │   ├── entity/         JPA-сущности (User, Image, Component, ...)
│   │   ├── repository/     Spring Data репозитории (DAO)
│   │   ├── dto/            DTO запросов/ответов
│   │   ├── service/        Бизнес-логика (Image/Component/Build/Auth)
│   │   ├── controller/     REST-контроллеры
│   │   ├── config/         Security, Async, CORS, DataSeeder
│   │   ├── filter/         JWT-фильтр
│   │   └── exception/      Глобальная обработка ошибок
│   ├── src/main/resources/application.properties
│   ├── pom.xml             Зависимости Maven
│   └── Dockerfile(.multistage)
├── frontend/               React + Vite
│   ├── src/api/            Axios-клиент и вызовы API
│   ├── src/store/          Zustand (состояние редактора)
│   ├── src/context/        AuthContext (токен, профиль)
│   ├── src/components/     Header, ImageCard, ComponentCard, ...
│   ├── src/pages/          Страницы (библиотеки, редакторы, auth)
│   ├── server.mjs          Статический сервер для Docker
│   └── Dockerfile(.multistage), vite.config.js
├── scripts/
│   ├── luckfox_build.sh    Обёртка сборки (симуляция/реальная)
│   ├── real-build.sh       Реальная сборка SDK с выводом в консоль
│   └── run-docker.sh       Сборка артефактов + docker compose
├── luckfox-pico/           SDK Luckfox (внешний, не в репозитории)
├── DOCs/                   Документация, диаграммы, схема БД
├── docker-compose.yml      db + backend + frontend
├── README.md
└── План_разработки_курсового_проекта.md
```

5. Модель данных и целостность

Реляционная модель MySQL. Основные таблицы и связи:

• **User** (id, name, surname, email, password) — таблица квотируется (User — зарезервированное слово).

• **Image** (id, name, description, distro, board, cost, state, artifact_path, build_log, version).

• **Component** (id, name, type→component_type, description, cost, version).

• **component_type** (id, type) — справочник типов.

• **dependency** (parent_id, dependent_id) — рекурсивные зависимости компонентов (M:N сам на себя).

• **image_component_dependency** (image_id, component_id) — состав образа (M:N).

• **user_image_order** (user_id, image_id) — заказы пользователя (M:N).

• **Role, Permission, user_role, role_premission** — роли и права (M:N).

Обеспечение целостности

• **Уровень БД:** PRIMARY/FOREIGN KEY, NOT NULL, уникальность email.

• **Транзакции** (@Transactional): атомарность операций над несколькими таблицами.

• **Оптимистичные блокировки** (@Version) на Image и Component — защита от потерянных обновлений.

• **Валидация** (Jakarta Bean Validation) на DTO: формат email, строгий пароль, обязательные поля.

• **Бизнес-правила:** граф зависимостей проверяется на циклы перед сборкой.

Рантайм-схему создаёт Hibernate (ddl-auto=update); файл **DOCs/этап3/java_working.sql** — документная модель данных.

6. Серверная часть (backend)

Сервисный слой

Класс	Ответственность
AuthenticationService	Регистрация (роль USER по умолчанию), вход, выдача JWT
JwtService	Генерация и валидация JWT-токенов
ComponentService	CRUD компонентов, поиск, фильтр по типу, проверка зависимостей
ImageService	Заказ сборки: транзитивное разрешение зависимостей, детекция циклов, привязка к пользователю
BuildService	Асинхронная сборка (@Async, один воркер): вызов скрипта, статусы, лог
BuildStateService	Транзакционные обновления статуса образа (QUEUED/BUILDING/READY/FAILED)
NotificationService	Уведомление о готовности образа (ссылка на скачивание)

Безопасность

• Stateless-аутентификация на JWT (HS256), фильтр JwtAuthenticationFilter.

• Пароли хранятся как BCrypt-хеши.

• Ролевая модель: @EnableMethodSecurity, мутации компонентов — только ROLE_ADMIN (@PreAuthorize).

• Чтение каталогов (GET образов/компонентов) — публично; заказ сборки и скачивание — только авторизованным.

• CORS настроен для фронтенда; глобальный обработчик ошибок возвращает аккуратные JSON-ответы.

7. REST API

Метод	Путь	Назначение	Доступ
POST	/api/auth/register	Регистрация	public
POST	/api/auth/login	Вход (JWT)	public
GET	/api/users/me	Профиль	auth
GET	/api/users/me/images	Заказы пользователя	auth
GET	/api/components	Каталог/поиск (?search=&type=)	public
GET	/api/components/{id}	Детализация компонента	public
POST	/api/components	Создать компонент	ADMIN
PUT	/api/components/{id}	Изменить компонент	ADMIN
DELETE	/api/components/{id}	Удалить компонент	ADMIN
GET	/api/component-types	Справочник типов	public
GET	/api/images	Библиотека образов	public
GET	/api/images/{id}	Образ и статус	public
POST	/api/images/build	Заказать сборку	auth
GET	/api/images/{id}/download	Скачать .img	auth

8. Очередь сборок и разрешение зависимостей

Очередь (один воркер)

Сборки обрабатываются строго последовательно: пул задач настроен на **один поток** (corePoolSize = maxPoolSize = 1, очередь до 1000 заказов). Пока собирается образ одного пользователя, заказы остальных ждут в статусе QUEUED и берутся по одному (FIFO). Это обязательно: реальная сборка SDK использует единый рабочий каталог, и параллельные сборки повредили бы друг друга.

Проверено: при трёх одновременных заказах один образ переходит в BUILDING, остальные остаются QUEUED и проходят последовательно на одном потоке build-worker-1 без пересечения по времени.

Разрешение зависимостей

• При заказе выбранные компоненты дополняются их транзитивными зависимостями (обход графа в ширину).
• Граф проверяется на циклы (DFS); при цикле заказ отклоняется с ошибкой.
• Стоимость образа = сумма стоимостей всех вошедших компонентов.

Запуск фоновой сборки выполняется строго после коммита транзакции (TransactionSynchronization), иначе асинхронный поток мог бы не найти ещё не сохранённый образ.

9. Клиентская часть (frontend)

SPA на React + Vite. Дизайн-токены (шрифт Inter, серо-«brand» палитра) вынесены в CSS-переменные.

Маршруты

Путь	Страница	Доступ
/login, /register	Вход и регистрация	public
/library/images	Библиотека образов (статусы, скачивание)	public
/library/components	Библиотека компонентов (поиск, фильтр)	public
/editor/image	Редактор образа (выбор компонентов, заказ)	auth
/editor/component	Редактор компонентов (создание/удаление)	ADMIN
/account	Профиль и мои образы	auth

Управление состоянием

• **Zustand** — состояние редактора образа (выбранные компоненты, дистрибутив, плата).
• **AuthContext** — токен и профиль пользователя.
• **Axios**-клиент с интерсепторами: подстановка Bearer-токена и обработка 401.
• Автообновление статусов сборок на страницах библиотеки/аккаунта (поллинг).

10. Тулчейн сборки Luckfox (симуляция и реальная)

Оркестрацию выполняет **scripts/luckfox_build.sh** в двух режимах:

Режим симуляции (по умолчанию)

• Собирает настоящий ext4-.img с rootfs-манифестом (/etc/luckfox-image.json) и списком пакетов.
• Позволяет проверить весь конвейер end-to-end без многочасовой кросс-компиляции.

• Не является загрузочным образом для железа — это артефакт-прототип.

Реальный режим (LUCKFOX_REAL_BUILD=1)

• Запускает SDK: ./build.sh (uboot + kernel + rootfs) и упаковку update.img.

• Требует развёрнутого SDK Luckfox и кросс-тулчейна; первый прогон ~1–3 часа.

• Выдаёт прошиваемые артефакты: uboot.img, boot.img, rootfs.img (ext4), update.img, sd_update.img.

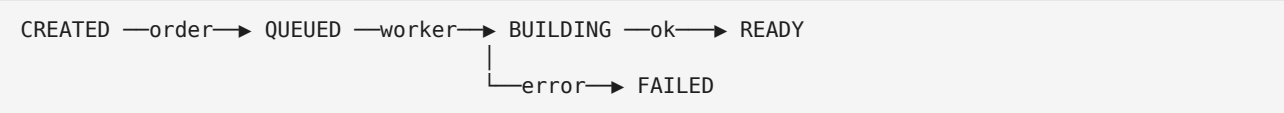
Решённые проблемы реальной сборки (WSL)

• **PATH с пробелами:** в WSL в $PATH подмешиваются Windows-пути (/mnt/c/Program Files/...), buildroot их отвергает. Решение: санитизация PATH в скриптах.

• **Зависание на мёртвых зеркалах:** wget без таймаута висел на недоступных mirror. Решение: ~/.wgetrc с таймаутами (connect/read 15/45с) — мёртвый mirror быстро отваливается на fallback.

Важно: структурная валидность артефактов подтверждена (реальная ext4-rootfs, корректные boot/uboot, валидный Rockchip update-пакет). Фактическая загрузка на железе требует прошивки на физический Luckfox Pico.

11. Жизненный цикл образа



Статус	Значение
CREATED	Образ создан (черновик)
QUEUED	Заказан, ждёт в очереди сборки
BUILDING	Воркер собирает образ
READY	Готов, .img доступен для скачивания
FAILED	Сборка завершилась ошибкой (см. build_log)

12. Развёртывание и запуск

Требования

• Java 17+, Maven

• Node.js 18+ и npm

• MySQL 8 (или через Docker)

• Docker и Docker Compose

• Для реальной сборки — SDK Luckfox (клонируется отдельно)

Вариант A — Docker (рекомендуется)

```
./scripts/run-docker.sh         # соберёт артефакты и поднимет стек
./scripts/run-docker.sh -d      # в фоне
./scripts/run-docker.sh down    # остановить
```

Фронтенд: http://localhost:5173 · Backend: http://localhost:8080 · MySQL: localhost:3306. Примечание: в WSL у Docker нет DNS внутри build-шагов, поэтому артефакты (jar и dist) собираются на хосте, а контейнеры их запускают (этим занимается run-docker.sh). Для сред с рабочим DNS есть Dockerfile.multistage.

Вариант B — локально

```
# 1) БД
docker compose up -d db
# 2) Backend
cd backend && mvn spring-boot:run
# 3) Frontend
cd frontend && npm install && npm run dev
```

Получение SDK Luckfox (для реальной сборки)

```
git clone https://github.com/LuckfoxTECH/luckfox-pico.git
```

Ключевые переменные окружения backend

Переменная	Назначение	По умолчанию
SPRING_DATASOURCE_URL	JDBC-строка к MySQL	jdbc:mysql://db:3306/luckfox_db
LUCKFOX_BUILD_SCRIPT	Путь к обёртке сборки	../scripts/luckfox_build.sh
LUCKFOX_OUTPUT_DIR	Каталог готовых .img	../output/images
LUCKFOX_REAL_BUILD	1 = реальная сборка SDK	0 (симуляция)
LUCKFOX_SIM_DELAY	Имитация длительности (сек, для демо очереди)	0

13. Учётные данные и проверка работоспособности

При первом старте DataSeeder наполняет каталог и создаёт администратора:

- **admin@luckfox.io / Admin@123** — роли ADMIN + USER (доступен редактор компонентов).

Правила валидации

- Email — стандартный формат (наличие @ и домена).
- Пароль — минимум 8 символов, хотя бы одна цифра, строчная и заглавная буквы и спецсимвол (@#$%^&+=!).

Что проверено end-to-end

• Авторизация: вход, отказ без/с битым токеном (403), ADMIN-only (403 для USER), слабый пароль и дубль email (400).

• Очередь сборок: последовательная обработка, QUEUED → BUILDING → READY.

• Полный конвейер: каталог → редактор → заказ (с авторазрешением зависимостей) → скачивание .img.

• Реальная сборка SDK: u-boot, kernel, rootfs и update.img собираются успешно.

14. Статус проекта и что доработать

Готово и проверено

• Backend: авторизация/роли, сервисы, разрешение зависимостей, очередь сборок, выдача .img.

• Frontend: библиотеки, редакторы, аккаунт, маршрутизация, состояние.

• Реальная сборка образа Luckfox (исправлены ошибки PATH и зависаний загрузок).

• Docker compose поднимает весь стек; план разработки переработан.

Осталось доработать

• **Маппинг «компонент → пакет buildroot (BR2_PACKAGE_*)»** — чтобы выбранные в UI компоненты реально попадали в образ (сейчас реальная сборка использует дефолтный набор).

• Включение реальной сборки в backend по умолчанию (монтирование SDK в контейнер, ресурсы, таймауты).

• Реальная доставка .img по email (сейчас — лог + ссылка на скачивание).

• Проверка загрузочности на физическом Luckfox Pico.

• Нагрузочное тестирование (JMeter) и кэширование каталога; вынос секретов из конфигов; тесты.

Документ сгенерирован автоматически из исходного кода и конфигурации проекта.